

Doc. no. P0181R1
Date: 2016-06-23
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Ordered By Default

Table of Contents

- [Revision History](#)
 - [Revision 0](#)
 - [Revision 0](#)
- [1 Introduction](#)
- [2 Stating the problem](#)
 - [Consistency of propagation traits](#)
- [3 Propose Solution](#)
- [4 Other Directions](#)
- [5 Formal Wording](#)
- [6 Acknowledgements](#)
- [7 References](#)

Revision History

Revision 0

Original version of the paper for the 2016 pre-Jacksonville mailing.

Revision 0

Include priority queue in proposal.

Clarify new functor is added to the `<functional>` header.

Add clause describing use of the `default_order` customization point.

Sketch of ideas to be turned into paper

This is a quick checklist of ideas that need to be incorporated.

- Write introduction
- Describe Problem - `operator<` is not the perfect default
- Propose Solution - indirection through default template parameter
- Describe any alternatives - `std2`
- Write Wording

1 Introduction

There are types we would like to store in standard associative containers that do not provide a natural `operator<`, but where a natural ordering can be defined. We would like to support them having a default order that is respected by the library, in an API and ABI compatible way that has no impact on existing code.

2 Stating the problem

The standard associative containers, and many algorithms, use `operator<` as the default choice for ordering sequences of arbitrary data types. This is a good choice for default, serving many types well, including all types in the standard library with the exception of `std::complex`. The question remains, how should a user create a type that deliberately does not want a mathematical ordering, like `std::complex`, but does want to be insertable into a `set` or used as the key to a `map` without requiring the user to explicitly specify the predicate to be used?

The common advice is that the user should specialize the `std::less` template for their purpose, but such a specialization then breaks the contract that `less` is a functor that calls `operator<`, and so such specializations are technically non-conforming, by not satisfying the original contract. This problem arises as we are requiring `std::less` to serve double-duty, as both the functor analog of the less-than operator, and the default mechanism to order arbitrary data types.

The conforming choice is to provide an implementation of `operator<` that is discovered through ADL. This works by breaking the original design of the user's class though, so cannot be recommended as the preferred direction.

What is needed is some other functor or function that serves as the default customization point to describe the ordering behavior of arbitrary types, that is typically, but not always, going to call `operator<`.

3 Propose Solution

The key observation is that `std::less` is not a part of the ABI of any of the standard containers, but merely a default template parameter. If the user explicitly specifies their own predicate, then `std::less` has no part to play. If the user does not specify a predicate and accepts the default, then `std::less` is substituted. Therefore, we can have a completely backwards compatible update, both API and ABI, as long as it guarantees to supply `std::less` as the default template parameter for any type that would support that today.

Falling back on the fundamental theorem of software engineering attributed to Andy Koenig, *We can solve any problem by introducing an extra level of indirection*. In this case, we want to add a type-computation to find the default ordering of an arbitrary type, an *ordering trait*, which defaults to `std::less<T>` but can be explicitly specialized by users for their own types to return a different default:

```
template <typename T>
struct default_order {
    using type = std::less<T>;
};

template <typename T>
using default_order_t = typename default_order<T>::type;

template <>
struct default_order<MyType> {
    using type = MyOrderingComparator;
};
```

We can then substitute this type-computation as the default template parameter for the predicate in associative containers:

```
template <class Key, class T, class Compare = default_order_t<Key>,
         class Allocator = allocator<pair<const Key, T>>>
class map;

template <class Key, class T, class Compare = default_order_t<Key>,
         class Allocator = allocator<pair<const Key, T>>>
class multimap;

template <class Key, class Compare = default_order_t<Key>,
         class Allocator = allocator<Key>>
class set;

template <class Key, class Compare = default_order_t<Key>,
         class Allocator = allocator<Key>>
class multiset;
```

This technique has been tried and tested on several popular compilers, including Clang, gcc, and Visual C++, and confirmed to produce identical symbols (ABI) as the current library.

The same problem is harder to solve for algorithms, as the standard algorithms do not expose a template-parameter functor computed by default. Instead, they simply provide two overloads, one that takes a predicate object, and one that defaults to using `operator<` directly in the implementation. That means that even if we have a default ordering for storing elements in a `set`, we are still unable to sort a vector of such elements without calling the `sort` overload that explicitly requires a predicate object.

The algorithm problem is still tractable, but messier. If we wish to solve this problem we need to add constrained overloads to the algorithms, so that types that can be ordered by `operator<` continue to be so ordered, but types that would otherwise fail to compile instead consult the `default_order` trait and forward to the predicate version of the algorithm instead, with a value-initialized predicate of the default ordered type. This can be achieved in C++14 using SFINAE hackery such as `enable_if`, but should be more cleanly specified in C++17 using concepts.

Note that the above formulation for algorithms would still lead to a discrepancy where types implement both a default order, *and* define on overloaded `operator<`. This could be refined by constraining on types there `is_same_v<default_order_t<T>, less<T>>`,

at the risk of ABI breakage. Note that such breakage is strictly opt-in on the part of the library author choosing to define a different default ordering for their type.

As the issues with the algorithms are a little more involved, this initial paper does not propose wording, although it should be fairly simple to provide in an updated paper, given suitable direction by the Library Evolution Working Group.

4 Other Directions

While writing this paper, alternatives for providing better defaults in general were considered. For example, replacing `std::less<T>` with `std::less<T>`. However, all such approaches lead to ABI breakage, and in some cases source breakage, so were deemed to be controversial to propose at this time. The author would encourage the Library Evolution Working Group to seriously consider providing *transparent* functors by default though, when the possibility of a compatibility-breaking library is under consideration.

The idea of simplifying the overload-sets of algorithms in the library by using default function template parameters was investigated for C++11 by Doug Gregor in [N2743](#), but this was rejected at the time for ABI breaking concerns. This is likely another topic worth revisiting at such time that the Library Evolution Working Group is prepared to consider a compatibility-breaking library, but not until.

5 Formal Wording

Make the following changes to the specified working paper:

20.13 Function Objects [function.objects]

Header <functional> synopsis

```
namespace std {  
    // ... elide header details and insert at the end  
  
    // 20.13.X, Default Functor Traits  
    template <class T = void>  
    struct default_order {  
        using type = std::less<T>;  
    };  
  
    template <class T = void>  
    using default_order_t = typename default_order<T>::type;  
}
```

20.13.X Default Functor Traits [func.default.traits]

```
namespace std {  
    template <class T = void>  
    struct default_order {  
        using type = std::less<T>;  
    };  
}
```

1. The class template `default_order` provides a trait that users can specialize for user-defined types to provide a strict weak ordering for that type, which the library can use where a default strict weak order is needed. For example, the associative containers and priority queue use this trait.
2. [Example:

```
namespace sales {  
    struct account {  
        int id;  
        std::string name;  
    };  
  
    struct order_accounts {  
        bool operator()(const Account& lhs, const Account& rhs) const {  
            return lhs.id < rhs.id;  
        }  
    };  
}  
  
namespace std {  
    template<>  
    struct default_order<sales::account>
```

```

        using type = sales::order_accounts;
    };
}

```

-end example 1

23.4.2 Header <map> synopsis [associative.map.syn]

```

namespace std {
    // 23.4.4, class template map:
    template <class Key, class T, class Compare = default_order_less<Key>,
              class Allocator = allocator<pair<const Key, T>>>
    class map;

    // 23.4.5, class template multimap:
    template <class Key, class T, class Compare = default_order_less<Key>,
              class Allocator = allocator<pair<const Key, T>>>
    class multimap;
}

```

23.4.3 Header <set> synopsis [associative.set.syn]

```

namespace std {
    // 23.4.6, class template set:
    template <class Key, class Compare = default_order_less<Key>,
              class Allocator = allocator<Key>>
    class set;

    // 23.4.7, class template multiset:
    template <class Key, class Compare = default_order_less<Key>,
              class Allocator = allocator<Key>>
    class multiset;
}

```

23.4.4.1 Class template map overview [map.overview]

```

namespace std {
    template <class Key, class T, class Compare = default_order_less<Key>,
              class Allocator = allocator<pair<const Key, T>>>
    class map {
        // ...
    };
}

```

23.4.5.1 Class template multimap overview [multimap.overview]

```

namespace std {
    template <class Key, class T, class Compare = default_order_less<Key>,
              class Allocator = allocator<pair<const Key, T>>>
    class multimap {
        // ...
    };
}

```

23.4.6 Class template set overview [set.overview]

```

namespace std {
    template <class Key, class Compare = default_order_less<Key>,
              class Allocator = allocator<Key>>
    class set {
        // ...
    };
}

```

23.4.7 Class template multiset overview [multiset.overview]

```

namespace std {
    template <class Key, class Compare = default_order_less<Key>,
              class Allocator = allocator<Key>>
    class multiset {
        // ...
    };
}

```

23.6.2 Header <queue> synopsis [queue.syn]

```
namespace std {
    template <class T, class Container = deque<T>> class queue;
    template <class T, class Container = vector<T>,
        class Compare = default_order_tless<typename Container::value_type>>
        class priority_queue;
}
```

23.6.5 Class template priority_queue [priority.queue]

```
namespace std {
    template <class T, class Container = vector<T>,
        class Compare = default_order_tless<typename Container::value_type>>
        class priority_queue {
            // ...
        };
}
```

6 Acknowledements

Thanks to Tony Van Eerd for repeatedly encouraging me to write this paper, and Howard Hinnant for encouraging experiments to confirm that the ABI claims hold up in practice.

7 References

- [N2743](#) Unifying Operator and Function-Object Variants of Standard Library Algorithms, Douglas Gregor